Reviewer Pack — Minimal Rank of Hodge Modules over Boolean Functions vs Mono…

Entry a64a6994734e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 02:15:23 UTC

Minimal Rank of Hodge Modules over Boolean Functions vs Monotone Circuit Size for k-CLIQUE

Entry ID: `a64a6994734e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 02:15:23 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

> {'stmt1': 'The minimal rank of a Hodge module associated with a Boolean function is upper bounded by the size of the smallest monotone circuit computing the function.', 'stmt2': 'For any Boolean function f, let H(f) be the Hodge module of degree k associated with f. Then, the minimal rank of H(f) satisfies:', 'stmt3': 'rank(H(f)) ≤ poly(n)'}

Rationale (proposer's reasoning):

> {'stmt1': 'Hodge theory provides a rich algebraic structure that can potentially capture complex properties of Boolean functions.', 'stmt2': 'The minimal rank of a Hodge module may serve as a quantitative measure for the complexity of computing the function using monotone circuits.', 'stmt3': 'Exploring this connection could lead to new insights into the inherent difficulty of computing certain functions in monotone circuits.'}

Taxonomy category: `MONOTONE_CLIQUE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5f2d1b115c4bc2da`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> If for at least 80% of the 30 random Boolean function instances, the minimal rank of the associated Hodge module is less than or equal to the size of the monotone circuit computing the function, then the conjecture is supported.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Rank Hodge Module Boolean Function Monotone Circuit Complexity` - `Hodge Theory Boolean function Monotone circuit size k-CLIQUE` - `Algebraic Geometry Hodge modules vs Monotone circuit complexity Boolean functions`

Top relevant hits considered: - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2005.05490v1] Monotone Boolean Functions, Feasibility/Infeasibility, LP-type problems and MaxCon - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [s2:f20309f4e51806ed0ff12d327139eba450b1c030] Computing and combinatorics : 15th annual international conference, COCOON 2009, Niagara Falls, NY, USA, July 13-15, 200

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def hodge_module_rank(f, k):
        # Placeholder function to compute the rank of a Hodge module
        # This is a dummy implementation and should be replaced with actual logic
        return len(f)

    def monotone_circuit_size(f):
        # Placeholder function to compute the size of a monotone circuit
        # This is a dummy implementation and should be replaced with actual logic
        return len(f)

    n = 40
    instances_tested = 30
    total_rank = 0
    total_circuit_size = 0
```

```
    for _ in range(instances_tested):
        f = generate_boolean_function(n)
        rank = hodge_module_rank(f, k=1)  # Assuming k is fixed at 1 for simplicity
        circuit_size = monotone_circuit_size(f)

        if rank > circuit_size:
            return {
                "metric_name": "rank(H(f))",
                "metric_value": rank,
                "instances_tested": instances_tested,
                "conjecture_holds": False,
                "counterexample": "Hodge module rank exceeds monotone circuit size"
            }

        total_rank += rank
        total_circuit_size += circuit_size

    avg_rank = total_rank / instances_tested
    avg_circuit_size = total_circuit_size / instances_tested

    return {
        "metric_name": "rank(H(f))",
        "metric_value": avg_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": avg_rank <= avg_circuit_size,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 1 for i in range(5, 8

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_rank = sum(r["metric_value"] for r in results)
    avg_rank = total_rank / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_rank} std=0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Hodge module rank exceeds monotone circuit size
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it was unable to complete the required number of instances to evaluate the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11953
2	preregistration	ollama_remote	glm4:latest	0	0	5356
3	novelty	ollama_remote	glm4:latest	0	0	4837
4	novelty	ollama_remote	glm4:latest	0	0	7287
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21739
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6512
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7617
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7781
9	judge	ollama_remote	glm4:latest	0	0	15138

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 88219 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a64a6994734e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a64a6994734e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a64a6994734e.tar.gz` (if generated)